Learn C++

Master C++ from absolute beginner to advanced system programmer

C++20 / C++23 Ready

Complete Learning Roadmap

25 Topics • 5 Sections • Beginner to Expert

Table of Contents

I. Fundamentals

1. Introduction to C++
2. Setup & Development Tools
3. Basic Syntax & Structure
4. Variables & Data Types
5. Operators
6. Control Flow

II. Core Concepts

7. Functions
8. Arrays & Strings
9. Pointers
10. References
11. Structures & Unions

III. Object Oriented

12. Classes & Objects
13. Inheritance
14. Polymorphism
15. Encapsulation
16. Abstraction

IV. Advanced

17. Templates
18. STL Containers
19. Memory Management
20. File I/O
21. Exception Handling
22. Modern C++

V. Expert

23. Multithreading
24. Networking
25. Design Patterns

01 Introduction to C++

History, features, and why learn C++

What is C++?

A high-performance, compiled language supporting procedural, object-oriented, and generic programming paradigms.

Beginner

History

Created by Bjarne Stroustrup in 1985 as 'C with Classes'. Evolved through C++98, C++11, C++14, C++17, C++20, and C++23.

Beginner

Use Cases

Game engines, operating systems, embedded systems, high-frequency trading, browsers, and performance-critical applications.

Beginner

```
// Your first C++ program
#include

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

02 Setup & Development Tools

Compilers, IDEs, and build systems

Compilers

GCC, Clang, MSVC. Understand compilation stages: preprocessing, compilation, assembly, linking.

Beginner

IDEs

Visual Studio, CLion, VS Code with C++ extensions, Qt Creator, Eclipse CDT.

Beginner

Build Systems

CMake, Make, Ninja, Bazel. Managing dependencies with vcpkg and Conan.

Intermediate

03 Basic Syntax & Structure

Program structure, comments, and naming conventions

Program Structure
Headers, main function, statements, semicolons, blocks, and return values.
Beginner

Comments
Single-line // and multi-line /* */ comments. Documentation with Doxygen style.
Beginner

Naming Conventions
CamelCase, snake_case, Hungarian notation. Consistency across codebases.
Beginner

04 Variables & Data Types
Built-in types, type modifiers, and constants

Fundamental Types
int, float, double, char, bool, void. Size and range of each type on different platforms.
Beginner

Type Modifiers
signed, unsigned, short, long, long long. Type aliases with typedef and using.
Beginner

Constants & Literals
const, constexpr, consteval. Integer, floating-point, character, and string literals.
Beginner

Auto & Type Inference
Using auto for type deduction. decltype and trailing return types.
Intermediate

```
const int MAX_SIZE = 100;
constexpr double PI = 3.14159;

auto count = 42;        // deduced as int
auto price = 19.99;     // deduced as double

unsigned long long big = 18446744073709551615ULL;
```

05 Operators
Arithmetic, logical, bitwise, and special operators

Arithmetic
+ - * / %. Integer division vs floating-point division. Precedence rules.
Beginner

Relational & Logical
== != < > <= >= && || !. Short-circuit evaluation.
Beginner

Bitwise Operators
& | ^ ~ << >>. Masking, setting, clearing, and toggling bits.
Intermediate

Special Operators
Ternary ?:, comma, sizeof, address-of &, dereference *, member access . ->.
Beginner

06 Control Flow

Conditionals, loops, and jump statements

Conditionals

if, if-else, if-else if-else, switch-case. Initializer if statements (C++17).

Beginner

Loops

for, while, do-while. Range-based for loops. Loop control with break and continue.

Beginner

Jump Statements

break, continue, return, goto. When to avoid goto.

Beginner

07 Functions

Declaration, definition, overloading, and recursion

Function Basics

Parameters, return types, pass-by-value, pass-by-reference. Default arguments.

Beginner

Function Overloading

Same name, different parameters. Name mangling and overload resolution.

Intermediate

Recursion

Base case and recursive case. Tail recursion optimization. Stack overflow risks.

Intermediate

Lambda Expressions

Anonymous functions, captures, mutable lambdas, generic lambdas (C++14).

Intermediate

```
// Function with default argument and reference parameter
void process(int& data, int multiplier = 1) {
    data *= multiplier;
}

// Lambda expression
auto square = [](int x) { return x * x; };

// Recursive factorial
constexpr int factorial(int n) {
    return (n <= 1) ? 1 : n * factorial(n - 1);
}
```

08 Arrays & Strings

Static arrays, dynamic arrays, C-style strings, and std::string

C-Style Arrays

Declaration, initialization, multidimensional arrays. Array decay to pointers.

Beginner

std::array

Fixed-size container with bounds checking. STL-compatible interface.

Beginner

std::vector

Dynamic arrays with automatic memory management. Capacity vs size.

Beginner

Strings

C-style strings (char*), std::string, string literals, string operations, std::string_view (C++17).

Beginner

09 Pointers

Memory addresses, pointer arithmetic, and smart pointers

Pointer Basics
Declaration, initialization, dereferencing, null pointers. Pointer to pointer.
Beginner

Pointer Arithmetic
Adding offsets, subtracting pointers, comparing pointers. Array-pointer equivalence.
Intermediate

Function Pointers
Declaring function pointers, callback mechanisms, std::function.
Advanced

Smart Pointers
unique_ptr, shared_ptr, weak_ptr. RAII and automatic memory management.
Intermediate

10 References

Lvalue references, rvalue references, and move semantics

Lvalue References
Aliases for existing variables. Pass-by-reference semantics. Must be initialized.
Beginner

Rvalue References
&& syntax. Binding to temporaries. Enabling move semantics.
Advanced

Move Semantics
std::move, move constructors, move assignment. Avoiding unnecessary copies.
Advanced

11 Structures & Unions

Aggregates, padding, and memory layout

struct
Grouping related variables. Member access. Nested structures.
Beginner

union
Shared memory layout. Type punning. Discriminated unions.
Intermediate

enum & enum class
Enumerated types. Strongly typed enums (C++11). Underlying types.
Beginner

12 Classes & Objects

The foundation of object-oriented programming in C++

Class Basics
Members, methods, access specifiers. this pointer. Default member initialization.
Beginner

Constructors
Default, parameterized, copy, move. Delegating constructors. Constructor initializer lists.
Intermediate

Destructors
Resource cleanup. Virtual destructors. Rule of Three/Five/Zero.
Intermediate

Operator Overloading
Overloading arithmetic, comparison, stream operators. Friend functions.
Intermediate

```
class Rectangle {
private:
    double width = 0.0;
    double height = 0.0;

public:
    Rectangle(double w, double h) : width(w), height(h) {}

    double area() const {
        return width * height;
    }

    // Operator overloading
    Rectangle operator+(const Rectangle& other) const {
        return Rectangle(width + other.width, height + other.height);
    }
};
```

13 Inheritance

Base and derived classes, access modes, and multiple inheritance

Single Inheritance
public, protected, private inheritance. Base class initialization.
Beginner

Multiple Inheritance
Inheriting from multiple base classes. Diamond problem and virtual inheritance.
Advanced

Virtual Functions
Dynamic dispatch. vtable and vptr. Cost of virtual functions.
Intermediate

14 Polymorphism

Runtime and compile-time polymorphism

Runtime Polymorphism

Virtual functions, override, final. Abstract classes and pure virtual functions.

Intermediate

Compile-time Polymorphism

Function overloading, operator overloading, templates.

Intermediate

RTTI

typeid, dynamic_cast. Runtime type information and its overhead.

Advanced

15 Encapsulation

Data hiding, getters/setters, and access control

Access Specifiers

public, private, protected. Friend classes and functions.

Beginner

Getters & Setters

Accessor methods. Const-correctness in accessors. Property patterns.

Beginner

PIMPL Idiom

Pointer to implementation. Hiding implementation details, reducing compile dependencies.

Advanced

16 Abstraction

Abstract classes, interfaces, and separation of concerns

Abstract Classes

Pure virtual functions (= 0). Cannot instantiate. Interface contracts.

Intermediate

Interfaces

Designing with interfaces. Dependency inversion principle.

Intermediate

Concepts (C++20)

Named requirements on template parameters. requires clauses.

Advanced

17 Templates

Generic programming with function and class templates

Function Templates
template. Type deduction. Explicit instantiation.
Intermediate

Class Templates
Generic classes. Template parameters, default template arguments.
Intermediate

Variadic Templates
Parameter packs. Fold expressions (C++17). Recursive unpacking.
Advanced

Template Metaprogramming
SFINAE, std::enable_if, constexpr if. Compile-time computation.
Advanced

18 STL Containers

Sequence, associative, and unordered containers

Sequence Containers
vector, deque, list, forward_list, array. Complexity guarantees.
Beginner

Associative Containers
set, map, multiset, multimap. Tree-based implementation. Custom comparators.
Intermediate

Unordered Containers
unordered_set, unordered_map. Hash tables. Custom hash functions.
Intermediate

Container Adaptors
stack, queue, priority_queue. Underlying container selection.
Beginner

19 Memory Management

Stack vs heap, new/delete, and memory safety

Stack vs Heap
Automatic storage duration. Dynamic allocation. Memory layout.
Beginner

new & delete
Dynamic allocation operators. Array allocation. Placement new.
Intermediate

Memory Leaks
Detection with Valgrind, AddressSanitizer. RAII as prevention.
Intermediate

Custom Allocators
Writing allocator-aware containers. Memory pools. PMR (C++17).
Advanced

20 File I/O

Streams, formatting, and binary I/O

Stream Classes
ifstream, ofstream, fstream. Opening modes. Error checking.
Beginner

Formatting
Manipulators, precision, width. std::format (C++20).
Beginner

Binary I/O
Reading/writing binary data. Serialization basics. Endianness.
Intermediate

21 Exception Handling

try, catch, throw, and exception safety guarantees

Basic Exception Handling
try-catch blocks. Throwing exceptions. Stack unwinding.
Beginner

Standard Exceptions
std::exception hierarchy. std::runtime_error, std::logic_error.
Beginner

Exception Safety
Basic, strong, no-throw guarantees. noexcept specifier.
Advanced

22 Modern C++

C++11/14/17/20/23 features

C++11 Essentials
auto, range-for, nullptr, constexpr, lambdas, smart pointers, move semantics.
Intermediate

C++17 Features
Structured bindings, if/switch init, inline variables, std::string_view, fold expressions.
Intermediate

C++20 Features
Concepts, ranges, coroutines, modules, spaceship operator <=>, designated initializers.
Advanced

C++23 Preview
std::expected, std::print, flat containers, deducing this.
Advanced

23 Multithreading

Concurrency, synchronization, and parallel algorithms

Thread Basics

std::thread, joining, detaching. Thread IDs. Hardware concurrency.

Intermediate

Synchronization

mutex, lock_guard, unique_lock, condition_variable. Deadlock prevention.

Advanced

Atomic Operations

std::atomic, memory ordering. Lock-free programming. ABA problem.

Advanced

Parallel Algorithms

Execution policies (C++17). std::execution::par. Performance considerations.

Advanced

24 Networking

Sockets, protocols, and C++ networking libraries

Berkeley Sockets

TCP/UDP sockets in C++. Blocking vs non-blocking I/O.

Advanced

Asio / Boost.Asio

Async I/O, timers, strands. Building scalable network applications.

Advanced

Networking TS

Standard networking library proposal. io_context, sockets, buffers.

Advanced

25 Design Patterns

Creational, structural, and behavioral patterns

Creational Patterns

Singleton, Factory, Builder, Prototype. Modern C++ implementations.

Advanced

Structural Patterns

Adapter, Decorator, Bridge, Composite, Facade, Proxy.

Advanced

Behavioral Patterns

Observer, Strategy, Command, Iterator, State, Visitor.

Advanced

Idioms

CRTP, Policy-based design, Type erasure. C++-specific techniques.

Advanced

Happy Coding! ■

Keep practicing and building projects to master C++.

C++ Learning Roadmap © 2026